

SalesVerse 2.0

Code Standards and Quality Report

Coding standards, static analysis, tests, coverage, dependencies and licences

Prepared for	Insurans Islam Family Takaful Sendirian Berhad
Solution	IIFT Agent/Banca Portal & Back-office
Build assessed	Commit 8651425
Evidence date	2 October 2026, 14:07 UTC
Document date	October 2026
Version	1.0
Classification	Commercial-in-Confidence

Commercial-in-Confidence. Prepared by iorta TechNXT for Insurans Islam Family Takaful Sendirian Berhad as part of the proposal for the Agent/Banca Portal and Back-office Solution.

Contents

Contents.....	2
1. Summary.....	3
2. Coding standards.....	4
CI pipeline gates.....	4
3. Static analysis.....	5
4. Automated tests.....	6
What the suites cover.....	6
5. Coverage.....	7
Coverage targets for implementation.....	7
6. Dependencies and licences.....	8
Vulnerability audit.....	8
Licence inventory.....	8
7. Code metrics.....	8
8. Maintainability.....	9
9. Known limitations and improvement plan.....	10
Annex A – Test titles.....	11
Annex B – Evidence files.....	13

1. Summary

This report describes the coding standards used to build SalesVerse 2.0 and the quality evidence for the current build (commit 8651425). All results are produced by tools, not by assessment, and the report is rebuilt from the evidence files each time they are refreshed.

Measure	Result
Lint (oxlint)	0 errors, 0 warnings
Type check (TypeScript compiler)	0 errors, 0 warnings
Formatting (Prettier)	0 errors, 0 warnings
API unit tests	54 of 54 passed
API end-to-end tests	29 of 29 passed
End-to-end line coverage	61.4%
Unit line coverage	14.51%
Dependency vulnerabilities (npm audit)	0 found
Runtime packages in licence inventory	494
Web lint (oxlint)	To be refreshed after front-end build
Web type check (tsc)	To be refreshed after front-end build
Web production build (Vite)	To be refreshed after front-end build

Table 1: Quality results

Static analysis, both test suites and the dependency audit are clean. End-to-end coverage is 61.4% of lines because those tests drive the real HTTP pipeline against PostgreSQL. Unit coverage is lower by design at this stage: unit tests target the calculation and rule code where defects are most costly. Section 5 sets the coverage targets for implementation.

Coding standards

The standards below apply to every change. Most are enforced by tools in the CI pipeline, so a change that breaks them cannot be merged.

Area	Standard	Enforced by
Language	TypeScript with strict, noImplicitReturns, noFallthroughCasesInSwitch, isolatedModules; target ES2023; module nodenext (ES modules on Node.js 22)	Type check in CI
Module structure	One NestJS module per functional area (22 modules: access, agency, aml, audit, auth, billing, claims, dashboard, documents, eod, esign, health, integration, integration-api, issues, notifications, participants, policies, products, reports, settings, workflow); shared code in src/common	Code review
Naming	Files in kebab-case with role suffix (.controller, .service, .dto, .guard, .spec); classes in PascalCase; database tables and columns in snake_case mapped from camelCase	Code review
Input validation	Every request body is a DTO class with class-validator rules; the global pipe uses whitelist, forbidNonWhitelisted and transform, so unknown fields are rejected	Global ValidationPipe; e2e tests
Data access	Prisma client only. 9 raw SQL statements use Prisma's tagged templates, which bind parameters; 0 use the unsafe string API	Code review; source scan in this report
Error handling	Business errors carry a code and a user message; one global filter returns status, code, message and correlation id, and logs full detail only on the server for 5xx	AllExceptionsFilter; security checks
Logging	Structured JSON logs (pino) with the request correlation id; cookies, authorisation and CSRF headers redacted; console logging forbidden	no-console lint rule
Comments	Comments explain why, not what; public services and guards carry a short doc comment, often citing the RFP requirement (for example AP-01/03/04)	Code review
Formatting	Prettier with single quotes, trailing commas 'all' and a line width of 100	prettier --check in CI
Linting (API)	eslint plugins typescript, unicorn, oxc, import; categories correctness=error, suspicious=warn; rules as errors: no-console, typescript/no-explicit-any, eqeqeq, no-eval	Lint step in CI
Linting (web)	eslint plugins typescript, react, react-perf, jsx-a11y, unicorn, oxc, import, including accessibility (jsx-a11y) and React hooks rules	Lint step in CI
Commits and review	Small commits with a descriptive message; every change through a pull request reviewed by a second engineer; the reviewer checks security, tests and RFP traceability	Pull-request review; branch protection on main
Branching	Short-lived feature branches from main; main is always releasable; releases are tagged	Branch protection

Table 2: Coding standards

CI pipeline gates

The pipeline runs on every pull request and on main. A failing step stops the merge.

Job	Step	Command or action
verify	npm ci	npm ci
verify	Generate database client	npx prisma generate
verify	Formatting	npx prettier --check "apps/*/src/**/*.{ts,tsx}" "apps/api/test/**/*..."
verify	Lint	npm run lint
verify	Type check	npm run typecheck --workspaces
verify	Unit tests	npm test --workspaces
verify	End-to-end API tests	npm run test:e2e --workspace apps/api
verify	Build	npm run build
verify	Dependency audit	npm audit --audit-level=moderate

Job	Step	Command or action
images	Build image from apps/api/Dockerfile	docker build -f apps/api/Dockerfile --target runtime -t salesverse-...
images	Build image from apps/web/Dockerfile	docker build -f apps/web/Dockerfile -t salesverse-web:\${{ github.sh...
images	Container vulnerability scan	aquasecurity/trivy-action@0.28.0

Table 3: CI steps (.github/workflows/ci.yml)

3. Static analysis

Run on 2 October 2026, 14:12 UTC with the same commands as the CI pipeline.

Check	Tool	Command	Errors	Warnings	Result
Lint	oxlint 1.86.0	npx oxlint src test	0	0	Pass
Type check	TypeScript compiler 6.0.3	npx tsc --noEmit -p tsconfig.json	0	0	Pass
Formatting	Prettier 3.9.9	npx prettier --check apps/api/src/**/*.ts apps/api/test/* */*.ts apps/api/prisma/**/*.ts	0	0	Pass

Table 4: API static analysis

Web application check	Result
Web lint (oxlint)	To be refreshed after front-end build
Web type check (tsc)	To be refreshed after front-end build
Web production build (Vite)	To be refreshed after front-end build

Table 5: Web application static analysis and build

4. Automated tests

Unit tests run with Vitest and need no database. End-to-end tests start the API with the production HTTP pipeline (the same middleware, guards, validation and error handling as the server) against a PostgreSQL 16 database, and call it over HTTP. Both suites run in CI on every change.

Suite	Tests	Passed	Failed	Run
API unit tests	54	54	0	2 October 2026, 14:04 UTC
API end-to-end tests	29	29	0	2 October 2026, 14:07 UTC

Table 6: Test results

What the suites cover

Test file	What it asserts	Tests	Passed
app-config.spec.ts	Start-up configuration: safe defaults, weak secrets rejected, insecure production settings refused	4	4
field-crypto.service.spec.ts	AES-256-GCM field encryption, tamper detection, blind index for identifiers	4	4
dates.spec.ts	Age at next birthday and Brunei business-day boundaries (UTC+8)	4	4
name-matching.spec.ts	AML fuzzy name matching: normalisation, honorifics, transliterations, thresholds	5	5
watchlist.spec.ts	Watch-list file parsing (quoted fields, CRLF)	1	1
password.service.spec.ts	Password policy rules, username check, history and expiry	6	6
file-inspector.spec.ts	Upload checks by content, forced extension, size limit	4	4
product-definitions.spec.ts	Product questionnaires: referral answers, completeness, unknown questions	4	4
reports.spec.ts	CSV formula neutralisation and report schedules in Brunei time	4	4
financing.engine.spec.ts	Mortgage Takaful rating (FTP-HP, FTP-NP, PFT) and the B\$150,000 referral	7	7
fixed-plan.engine.spec.ts	Annual plan rating (PHA, PRO, KHR, OSA), eligibility and riders	11	11
access-control.e2e-spec.ts	Authentication, audience separation, permissions and data scoping over HTTP	11	11
auth.e2e-spec.ts	Sign-in, cookie flags, logout, password change, logout and session revocation	7	7
operations.e2e-spec.ts	Onboarding, AML review, issues with SLA and other back-office journeys	8	8
policy-lifecycle.e2e-spec.ts	Quotation to policy with maker-checker payment, agency block, referral approval	3	3

Table 7: Test files

Every test title is listed in Annex A.

5. Coverage

Measure	Unit tests	End-to-end tests
Lines	14.51%	61.4%
Statements	14.93%	61.23%
Functions	10.59%	62.15%
Branches	18.12%	51.23%

Table 8: Overall coverage

Module or area	Lines	Unit	End-to-end
access	139	0.0%	58.3%
agency	230	0.0%	37.8%
aml	194	37.1%	56.2%
app.module.ts	3	0.0%	66.7%
app.setup.ts	18	0.0%	94.4%
audit	21	4.8%	90.5%
auth	212	14.6%	83.0%
billing	144	0.0%	53.5%
claims	71	0.0%	73.2%
common	172	32.6%	83.1%
config	79	86.1%	68.4%
dashboard	45	0.0%	91.1%
documents	223	9.9%	74.4%
eod	79	0.0%	83.5%
esign	78	0.0%	42.3%
health	19	0.0%	36.8%
integration	128	0.0%	75.0%
integration-api	55	0.0%	49.1%
issues	111	0.0%	69.4%
main.ts	3	0.0%	0.0%
notifications	84	0.0%	57.1%
participants	83	0.0%	49.4%
policies	366	0.0%	56.6%
products	59	40.7%	62.7%
products/rating (rating engines)	193	83.9%	71.5%
reports	232	15.1%	84.5%
settings	85	4.7%	65.9%
workflow	146	0.0%	67.1%

Table 9: Line coverage by module

Coverage targets for implementation

Coverage is reported honestly rather than inflated with low-value tests. The rating engines already carry focused unit tests; most services are covered only through end-to-end journeys. During implementation we add service-level unit tests and SIT automation, with these targets:

Area	Target	Measured by
Rating engines (products/rating)	At least 80% lines and branches	Unit tests, per release
Billing (payments, grace period)	At least 80% lines	Unit plus end-to-end
Workflow (maker-checker)	At least 80% lines	Unit plus end-to-end
All API code	At least 70% lines combined	Merged unit and end-to-end coverage in CI

6. Dependencies and licences

Vulnerability audit

npm audit reports 0 known vulnerabilities across 933 installed packages (514 production, 405 development, 136 optional). CI fails on any moderate or higher finding, and a container scan (Trivy) fails the image build on any fixable Critical or High finding.

Severity	Critical	High	Moderate	Low	Info	Total
Findings	0	0	0	0	0	0

Licence inventory

The runtime dependency tree contains 494 packages. All carry permissive open-source licences apart from the items explained below; none imposes copyleft obligations on the deployed solution.

Licence	Packages
MIT	400
ISC	40
Apache-2.0	30
BSD-3-Clause	7
Unlicense	3
UNKNOWN	2
MIT/X11	2
BSD-2-Clause	2
MIT and ISC	1
Python-2.0	1
EPL-2.0	1
(MIT OR GPL-3.0-or-later)	1
MIT-0	1
(MIT AND Zlib)	1
0BSD	1
MIT AND ISC	1

Table 10: Packages by licence

Package	Version	Declared licence	Position
buffers	0.1.1	UNKNOWN	MIT licence declared in the project repository but not in package.json.
elkjs	0.11.1	EPL-2.0	Reached only through the Prisma CLI's optional Studio peer, a development tool. It is not installed in the runtime image, which is built with npm ci --omit=dev.
jszip	3.10.2	(MIT OR GPL-3.0-or-later)	Dual-licensed MIT OR GPL-3.0; used under the MIT licence.
png-js	2.0.0	UNKNOWN	MIT licence declared in the project repository but not in package.json.

Table 11: Licence exceptions and how they are handled

7. Code metrics

Measure	Value
API source files	142
API source lines (code / comment / blank)	15,008 / 518 / 1,153
End-to-end test files and lines of code	7 files, 867 lines

Measure	Value
Unit test files	11
Seed and reference data (files / lines of code)	6 / 1,791
Functional modules	22
API paths / operations	128 / 149
Database tables / enumerations	37 / 36
Schema lines / migration SQL lines	915 / 883

Table 12: Size metrics (API)

Generated code (the Prisma client in src/generated) is excluded from linting and from these counts.

8. Maintainability

Each functional area is a separate NestJS module with its own controller, services and DTOs. Modules talk through injected services, not shared tables, so an area can be changed, tested or later moved to its own service without touching the others.

Business behaviour that IIFT will want to change is configuration, not code. Products, plans and rate tables, questionnaires, workflow definitions and approval thresholds, security and business parameters (password policy, lockout, session limits, grace period, SLA hours) and master data are maintained in the back-office under maker-checker and take effect without a release.

The API, the web tier and the database schema are versioned together. Database changes are applied by versioned SQL migrations, and the same container images move from SIT to UAT to production.

9. Known limitations and improvement plan

Area	Current position	Action during implementation	When
Service-level unit coverage	Unit tests concentrate on rules and calculations; most services are exercised through the end-to-end suite only.	Add unit and SIT automation; reach at least 80% line coverage on rating, billing and workflow modules	Weeks 6–19
Front-end evidence	Lint, type-check and build evidence for the web application is captured separately.	Refresh this report after the front-end build; add Playwright smoke tests to CI	Weeks 6–16
Integration adapters	Core, FIN, AML provider and SMS adapters run in simulated mode until IIFT endpoints are available.	Contract tests per interface specification; joint tests with system owners	Weeks 8–17
Performance evidence	No load test results yet.	k6 load, stress and soak tests on production-like SIT (DEL-18)	Weeks 17–18
Mutation and accessibility checks	Not yet part of CI.	Add an accessibility check on key screens and a periodic mutation-test run on rating engines	Weeks 10–19

Table 13: Improvement plan

Annex A – Test titles

Test file	Test	Result
app-config.spec.ts	loads a minimal development configuration with safe defaults	Passed
app-config.spec.ts	fails fast on missing or weak secrets	Passed
app-config.spec.ts	refuses insecure production settings	Passed
app-config.spec.ts	accepts a complete production configuration	Passed
field-crypto.service.spec.ts	round-trips a value and never produces the same ciphertext twice	Passed
field-crypto.service.spec.ts	detects tampering through the authentication tag	Passed
field-crypto.service.spec.ts	produces the same blind index for differently formatted identifiers	Passed
field-crypto.service.spec.ts	normalises and masks identifiers	Passed
dates.spec.ts	computes age at next birthday	Passed
dates.spec.ts	uses the Brunei calendar day (UTC+8) as the business date	Passed
dates.spec.ts	returns the UTC instants bounding a Brunei business day	Passed
dates.spec.ts	adds months and counts days	Passed
name-matching.spec.ts	normalises case, punctuation, honorifics and common spellings	Passed
name-matching.spec.ts	scores identical names 100 regardless of honorific	Passed
name-matching.spec.ts	scores close transliterations above the default review threshold	Passed
name-matching.spec.ts	scores unrelated names low	Passed
name-matching.spec.ts	implements Jaro-Winkler	Passed
watchlist.spec.ts	handles quoted fields, escaped quotes and CRLF line endings	Passed
password.service.spec.ts	accepts a password meeting every rule	Passed
password.service.spec.ts	reports every unmet rule together	Passed
password.service.spec.ts	rejects passwords containing the username	Passed
password.service.spec.ts	hashes with Argon2id and verifies	Passed
password.service.spec.ts	expires passwords after the configured number of days	Passed
password.service.spec.ts	generates temporary passwords that satisfy the policy	Passed
file-inspector.spec.ts	identifies files by content and forces a matching extension	Passed
file-inspector.spec.ts	rejects content that is not PDF, PNG or JPEG whatever its name	Passed
file-inspector.spec.ts	enforces the size limit and requires a file	Passed
file-inspector.spec.ts	builds safe file names	Passed
product-definitions.spec.ts	returns referral reasons for "yes" answers that require underwriting	Passed
product-definitions.spec.ts	requires every question to be answered and details for "yes"	Passed
product-definitions.spec.ts	rejects answers to questions the product does not ask	Passed
product-definitions.spec.ts	treats documents as mandatory unless stated otherwise	Passed
reports.spec.ts	quotes CSV cells and neutralises spreadsheet formulas	Passed
reports.spec.ts	runs daily reports at 07:00 Brunei time (23:00 UTC)	Passed
reports.spec.ts	runs weekly reports on Monday and monthly reports on the 1st (local time)	Passed
reports.spec.ts	resolves relative periods to date ranges	Passed
financing.engine.spec.ts	calculates a decreasing-term contribution on the flat-rate financing total	Passed
financing.engine.spec.ts	refers financing above the B\$150,000 high-risk limit to IIFT Sales	Passed
financing.engine.spec.ts	does not refer financing exactly at the high-risk limit	Passed
financing.engine.spec.ts	rejects cover that would run past the maximum age at expiry	Passed
financing.engine.spec.ts	applies no profit loading when the product uses NONE as profit basis	Passed
financing.engine.spec.ts	lists every missing risk field at once	Passed
financing.engine.spec.ts	validates its configuration	Passed
fixed-plan.engine.spec.ts	prices Professional Plan B with the optional additional cover (FFR03)	Passed
fixed-plan.engine.spec.ts	accepts only Occupational Class I for the Professional plan	Passed
fixed-plan.engine.spec.ts	applies the Wider (Child) loading for Khairat (FFR04)	Passed
fixed-plan.engine.spec.ts	requires a coverage type when the product defines them	Passed
fixed-plan.engine.spec.ts	offers 1- and 2-year cover for Personal Home Assistant (FFR02)	Passed
fixed-plan.engine.spec.ts	accepts a registered Brunei citizen student	Passed
fixed-plan.engine.spec.ts	rejects a participant who is not a Brunei citizen	Passed
fixed-plan.engine.spec.ts	requires confirmation of student registration	Passed
fixed-plan.engine.spec.ts	rejects participants older than 65 next birthday	Passed
fixed-plan.engine.spec.ts	drops risk details that the product does not define	Passed
fixed-plan.engine.spec.ts	validates every catalogue product configuration	Passed
access-control.e2e-spec.ts	rejects unauthenticated requests	Passed

Test file	Test	Result
access-control.e2e-spec.ts	keeps portal users out of the back-office API and vice versa	Passed
access-control.e2e-spec.ts	enforces permissions within the back-office	Passed
access-control.e2e-spec.ts	restricts records to the agent hierarchy and agency	Passed
access-control.e2e-spec.ts	treats injection attempts in search parameters as plain text	Passed
access-control.e2e-spec.ts	rejects unknown and malformed fields	Passed
access-control.e2e-spec.ts	never exposes internal error details and returns a correlation id	Passed
access-control.e2e-spec.ts	sends security headers and hides the framework	Passed
access-control.e2e-spec.ts	authenticates system-to-system calls with an API key	Passed
access-control.e2e-spec.ts	stores identification numbers encrypted and shows them masked	Passed
access-control.e2e-spec.ts	keeps the audit trail append-only at database level	Passed
auth.e2e-spec.ts	rejects wrong credentials with a generic message	Passed
auth.e2e-spec.ts	issues an HttpOnly, SameSite=Strict session cookie and never returns secrets	Passed
auth.e2e-spec.ts	locks the account after the configured number of failed attempts	Passed
auth.e2e-spec.ts	requires the CSRF token on state-changing requests	Passed
auth.e2e-spec.ts	ends the session on logout	Passed
auth.e2e-spec.ts	keeps only one active session per user	Passed
auth.e2e-spec.ts	forces a temporary password to be changed before anything else	Passed
operations.e2e-spec.ts	onboards a sub-agent through registration, AML, documents and approval (AP-07, BO-10/13)	Passed
operations.e2e-spec.ts	routes a watch-list match to Compliance and records the review (AP-16, BO-13..15)	Passed
operations.e2e-spec.ts	manages an issue through assignment, comments and resolution with SLA dates (AP-55..57, BO-29..31)	Passed
operations.e2e-spec.ts	accepts claim notifications only within the period of cover (AP-43)	Passed
operations.e2e-spec.ts	runs every standard report and exports it in each format (BO-22..24)	Passed
operations.e2e-spec.ts	runs end-of-day, posts to FIN, reconciles and delivers integration messages (FFR EOD, INT-13..15)	Passed
operations.e2e-spec.ts	administers users, roles, parameters, master data and workflows with an audit trail (BO-03/04/26..33, COM-04)	Passed
operations.e2e-spec.ts	serves the dashboards (AP-52/53, BO-20/21)	Passed
policy-lifecycle.e2e-spec.ts	runs the full Professional Takaful Plan journey with maker-checker payment verification	Passed
policy-lifecycle.e2e-spec.ts	blocks new business for an agency with overdue contributions (AP-42)	Passed
policy-lifecycle.e2e-spec.ts	refers high-risk financing for approval and prevents self-approval	Passed

Table 14: All automated tests

Annex B – Evidence files

Results in this report are read from the evidence files below when the report is built. The fingerprint is the start of each file's SHA-256 hash, so a reader can confirm which evidence a given version of the report was built from.

Evidence file	Size	SHA-256 (first 16 characters)
api-e2e-coverage-summary.json	46,528 bytes	6c079f523f5bfac0...
api-e2e-tests.json	933,978 bytes	42706cfb62722b95...
api-static-analysis.json	796 bytes	c089c515ae4ee33b...
api-unit-coverage-summary.json	42,137 bytes	be2af8466b49500f...
api-unit-tests.json	887,671 bytes	fa135bc56e5fab94...
code-metrics-api.json	759 bytes	2ca7d0802f7f1c9d...
npm-audit.json	366 bytes	1cf279e9ef691d09...
runtime-dependency-licences.json	40,391 bytes	b003a0dc1a75e06f...
security-checks.json	9,632 bytes	a1ce57e45ec687c6...

Table 15: Evidence files